Language-independent Development Environment Support for Dynamic Runtimes

Anonymous Author(s)

Abstract

There are many factors for the success of a new programming language implementation. Existing language implementation frameworks such as Truffle or RPython have focused on run-time performance and security, or on providing a comprehensive set of libraries.

The tools that language users need to be productive are also an important factor for the adoption of a programming language. The aforementioned frameworks, however, provide limited support for creating comprehensive development tools. This is an impediment to language adoption that can be as serious as performance issues or lack of libraries. Both Truffle and RPython already provide run-time tools such as debugging, profiling, or coverage tools in a language-independent manner, but neither support static development tasks carried out in code editors.

In this work, we propose an approach to ~~close the tooling gap of new language implementations~~ add this missing tooling by making the Language Server Protocol available automatically to all language implementations on the Truffle framework. Furthermore, we show how our approach can utilize run-time information to provide ~~Integrated Development Environment~~ IDE features rarely available for dynamic programming languages.

1 Introduction

As use-cases evolve, so do programming languages to deal with the rising complexity of modern software systems. Developing a new programming language from scratch is a challenging task. Traditionally, ensuring the performance of new implementations is competitive with existing ones has consumed a large amount of development time. Frameworks such as RPython [1] and Truffle [23] address this and provide good performance with reasonable effort on the part of the language developer.

However, another barrier for new and evolving languages has been tool support. Users often have to give up on existing tools and learn new ones, if there even are replacements for all the tools they were used to when switching to a new language. The ~~Language Server Protocol (LSP) tackles the problem of tool development~~ Truffle and RPython frameworks have recognized this and thus provide hooks for language implementations to gain support for interactive debugging, profiling, and coverage information easily.

One tool that's still missing is a code editor with a rich set of capabilities. The LSP tackles this tooling aspect by connecting multiple Development Environments (DEs) with a *language server*. This server is implemented per language, but the burden of building or integrating with existing environments is handled by the protocol.

In this work, we propose an approach to provide ~~tooling~~ assisted code editing for new language implementations in a framework like Truffle with the same ease as it provides performance. Language implementers merely have to use the hooks provided by the framework in order to get an integration with any LSP client for free, and receive at least basic syntax error reports, completion of globals and locals, as well references, definition, and meta-information lookup. Our generic approach is not meant to replace established language servers for a specific language, but provide a good baseline for languages for which no such server exists, yet.

~~Another challenge for Integrated Development Environments (IDEs) especially~~ Code assistance for dynamic language code is especially challenging and our approach includes suggestions on how to collect and utilize run-time information to improve editing features for dynamic languages~~is support for features like code completion, goto-definition, or finding all method references~~. For programming languages with typed symbols, like Java, the required information can be derived from the source code, but for dynamically typed languages, IDEs typically try to infer types and/or make use of additional type hints in source code comments or other annotations. Good type inference engines are difficult to build and require ~~deep~~ language-specific knowledge. Many dynamic languages also include constructs ~~which prevent any kind of~~ that prevent type inference such as `eval`.

~~Our approach also includes suggestions on how to collect and utilize run-time information, to improve certain IDE features for dynamic languages.~~ As proof of concept, we have implemented Graal-lsp, a polyglot language server implementation for the GraalVM.

Thus, our contributions in this work are:

- An approach to build a language-agnostic language server on top of performance-focused language implementation frameworks.
- An approach to provide IDE-like features for dynamic languages, by collecting and utilizing run-time information.

PL'18, January 01–03, 2018, New York, NY, USA
2018.

- An implementation of these approaches using the Truffle framework, which includes polyglot code completion and goto-definition possibilities.

The rest of this paper is structured as follows: in section 2, we provide background information pertaining to the LSP and language implementation frameworks. In section 3 we describe our approach to build a language-agnostic language server on top of a run-time focused language implementation framework. In we give an overview of our prototype. Truffle and what the language developer has to do in order to support the different features. Finally, in section 4 we discuss related work and we conclude and summarize future work in section 5.

2 Context

In this section we give a brief overview over the two areas we combine in this work: a) the Language Server Protocol (LSP) , and b) language implementation frameworks with Truffle and RPython as concrete instances .

2.1 Code Editors and the Language Server Protocol

A huge number of code editors exist and the choice of editor is an amusingly frequent source of arguments amongst developers. The Language Server Protocol (LSP) [1] is an approach to standardize language support for development tools to make the choice of editor less tied to the choice of language. The LSP defines a *server* and a *client*. The server needs to be implemented only once for every language, providing the language-specific development features. Any editor that can act as a language client can make use of any language server and therefore support multiple languages with minimal effort.

An exemplary communication of language client and server, including initialization, opening and changing a file, and requesting a code completion and goto-definition.

shows an exemplary communication between server and client. After initialization, the client notifies the server that a file has been opened. The server parses the source code, detects a syntax error and sends this information to the client. The user fixes the displayed syntax error and the client sends these changes incrementally to the server. After parsing again on the server side, no errors occur, but the server needs to explicitly notify the client about that, so that the client can clear the currently displaying diagnostics. Then, the client sends a request for code completion at a specific source location, typically the caret position. The server obtains completion items depending on the provided context, which might also contain the trigger character, and sends the items to the client. Later, a request to find one or more definitions for a symbol at a position in the source code is sent to the server. The server responds with a list of locations, if found.

In this work we want to focus on some of the high-level features the LSP supports and that most editors afford to their users:

Navigational Features These are the features code editors provide to navigate through source code. In addition to pure textual searches, there might be a hierarchical outline views or actions to find all definitions or references of a symbol.

Informational Features These features visualize source code related information, e.g. to display signature and documentation, type information, or highlight uses of a symbol in the same compilation unit. If a language parser detects syntax errors or a linter provides some warnings, this information is also displayed in the code editor.

Code-modifying Features This includes actions to format the whole file or a code snippet according to some formatting rules as well as refactoring support. (Renaming of a symbol in the current file or all files in the workspace is a simple example.) Sophisticated refactoring tools, however, are rather found in IDEs than source code editors.

Code Completion Code completion is an important feature and typically integrated seamlessly into the textual editor. We describe this feature in more detail here, because we will use it as a running example in .

Code completion when triggered after object navigation characters (i.e. the *dot* in Java) includes names of the methods and fields declared by the object's type and super types before the navigation character. This kind of code completion we call . For non-object-oriented languages, might not be supported or be limited to special language elements, which understand some kind of *property* semantic. is often a challenge for languages with untyped symbolsinstances of performance-focused language implementation frameworks, and b) the Language Server Protocol (LSP) and how we approached supporting it in a language-independent manner.

Another kind of code completion we will call . In this case, the completion items depend on the surrounding scope rather than the symbol in front of the cursor. With *globals* we refer to all the symbols which we can derive from global scopes of languages. With *locals* we analogously refer to all symbols which we can derive from local scopes of languages.

Because code completion is a core feature and is frequently auto-activated while typing, Asaduzzaman has collected requirements of code completion systems [2]. Most of them address the fact, that the completion system should not interrupt the user's flow of code writing.

[1] (accessed 2018-12-17). At the date of writing, language servers for more than 50 languages exist. Usually, these language server implementations support exactly one language, but the LSP allows multiple languages per server.

2.1 ~~Language Implementation Frameworks~~

~~Language Implementation Frameworks help a language developer in the process of creating a new language. The feature set varies immensely, depending on the purpose of the framework. Language workbenches like Spoofax [10], Xtext [6], or MPS [22], are used to develop and compose Domain Specific Languages (DSLs) together with a corresponding IDE with minimal effort, but their focus is not on general purpose languages with good performance.~~

~~The kinds of frameworks~~

2.1 Tools in Language Implementation Frameworks

In this work we are interested in ~~in this work are~~ performance-focused language implementation frameworks like Truffle [23] or RPython [1]. In these frameworks, tooling has been a focus only where it did not compete with performance~~, and thus~~. Thus, languages developed in these frameworks have fewer tools available when compared to language workbenches like Spoofax [10], Xtext [6], or MPS [22], which are used to create DSLs and their corresponding IDEs.

~~Truffle and GraalVM~~ Truffle [23] is a language implementation framework written in Java. The language developer uses the framework to write an Abstract Syntax Tree (AST) interpreter, which ~~is able to optimize itself~~ the framework optimizes during interpretation. ~~To achieve that, Truffle uses run-time type information to rewrite AST nodes.~~ How source code ~~is~~ parsed and the AST is structured is the responsibility of the language implementation.

Truffle has built-in support for developing language-agnostic tools by providing an instrumentation ~~Application Programming Interface (API) [18] and a system of tags and source information~~ API [18] using a meta-information that language implementations can attach to their AST nodes. ~~It is designed to work well with the optimization techniques for language implementations so that instrumentation can be done with nearly zero-overhead. The framework can notify tools when a new source is loaded, a statement is going to be executed, or an expression will return a value. The focus of this instrumentation has been to build run-time tools like debuggers and profilers.~~

~~To get high-performance language implementations, Truffle uses the language-independent just-in-time compiler named Graal [25]. Among other optimizations, Graal applies partial evaluation to compile stable AST subtrees into optimized machine code [24]. These optimizations do not stop at language boundaries, so that the access of foreign language objects can also be optimized~~This APIhas been used already to provide all Truffle languages with code coverage, CPU and memory profiling tools, as well as debugging with the Chrome debugger protocol that's also supported by the VSCode, IntelliJ, and Netbeans.

~~Truffle languages and tools hosted by the Truffle framework on top of the GraalVM.~~

~~PyPy and RPython~~ PyPy [15] is an alternative implementation of the Python [19] language ~~. It uses a tracing Just-in-time (JIT) compiler [3] so that many Python programs can be executed faster than using the default Virtual Machine (VM) implementation *CPython*.~~

~~RPython[1] is a translation and support framework for developing dynamic programming languages. The RPython language is~~ written using RPython, a restricted subset of ~~Python. It disallows some of the dynamic features of Python to apply static analysis. Thus, source code written in RPython can be translated to C by deriving low-level C types from Python data types. Language implementers use the framework to write a bytecode interpreter with hints for the RPython tool chain, so that the interpreter can be optimized by PyPy's tracing JIT compiler.~~

the Python language. The language developer is free to structure the interpreter as they see fit, although bytecode interpreters are the most common approach for RPython-based languages. RPython provides no ~~explicit API for tools as described for the Truffle framework. The only notable example of a run-time tool is VMProf, a profiler to understand and resolve performance bottlenecks. The language developer registers~~ generic APIfor tools like Truffle does. However, the RPython library includes a CPU profiler and a reverse debugger. Languages can make use of these with minimal effort by registering entry points for the ~~profiler by annotating functions~~ with certain Python decorators and by implementing ~~a callback to get string representations of the guest language objects~~callbacks to attach for the tools to call into.

3 ~~Approach~~

2.1 Code Editors and the Language Server Protocol

~~Typically, a language server is developed for one specific language only and the server implementation can exploit any kind of~~ A huge number of code editors exist and the choice of editor is an amusingly frequent source of arguments amongst developers. The Language Server Protocol (LSP)[1] is an approach to standardize language support for development tools to make the choice of editor less tied to the choice of language. The LSP defines a *server* and a *client*. The server needs to be implemented only once for every language, providing the language-specific ~~information. To implement~~

[1] https://microsoft.github.io/language-server-protocol (accessed 2018-12-17). At the date of writing, language servers for more than 50 languages exist. Usually, these language server implementations support exactly one language, but the LSP allows multiple languages per server.

a language-agnostic language server, we have to build upon the abstract concepts and APIs of language implementation frameworks, which apply to all languages developed in these frameworks. In the following, we use the example of code completion to describe our approach. First of all, we show which kind of data is needed to respond to the *completion* request of LSPdevelopment features. Any editor that can act as a language client can make use of any language server and therefore support multiple languages with minimal effort. Then, we suggest how languageimplementation frameworks can obtain the needed information in a language-agnostic way.

2.2 Completion of Globals and Locals

In we distinguished two kinds of code completions. We start with the Python code snippet in to explain how to support code completion of globals and locals.

```
def foo(param1, param2): x = 42
y = param1 + param2
print(x * y)
```

With the caret on line 3, a *completion* request is sent to the server for line 3 at character offset 4. We would expect a completion result to include items for all Python globals (e.g. built-in functions) and the locals , , and . The variable should not be included, because no value is assigned to it until line 4. Besides a display name, a LSP completion item can optionally contain a completion kind (e.g. variable, function, or class), a detail text for signature or type information, and a documentation. To provide these items, we suggest the following steps. For a number of Truffle languages (including R, Python, Ruby, and JavaScript), multiple LSP server implementations already exist and these can be used with these implementations directly. However, both Truffle already tries to obsolete the need for multiple implementations of a GC, JIT, dynamic object model, debugger, profiler, and more. We argue that there should not be a need for multiple implementations of language servers, either.

In this work we focus on how Truffle can (in a language independent manner) support a subset of the features of the LSP: 1) Parse the source code into a language-agnostic Intermediate Representation (IR) of the framework. This just re-uses the parser that the language developer has to write anyway. This parser will typically be syntax-error intolerant, so that we can support code completion only if the source code is parsable. navigational features to follow references of names and go to the implementations of functions; 2) Map line and column of the completion request to the IR. To do this, we assume selected elements of the IR allow access lexical representations in the source code. This information allows us to search for an IR element which maps best to the position of the completion request. Which one that should be in particular depends on the IR and the framework. We therefore assume that language implementations can expose further meta-information in a language-agnostic way through the framework, so that we can determine relevant language elements, like read- and write-accesses to variables, calls to callable elements, or definitions. informational features to display syntax errors, documentation, function signatures, and types; 3) Derive lexical scopes from the mapped IR-element. Typically, there are some language built-ins which are always available, independent of the current context, which a language implementation can just return through a hook as global scope. Another hook can be used by the language to match the selected IR element to a local scope. code completion both of names available in local and global scopes as well as properties on names or results of operations; and 4) Build completion items by gathering additional information for the obtained symbols of the scopes. Depending on the meta-information available for each symbol, we can add a completion kind, signature information for callables, or a documentation text. Due to the dynamic nature of our targeted languages , this information will be typically not available at parse-time. code completion when multiple languages are mixed as an example of a feature that other language servers cannot easily support.

2.2 Obtaining and Using Run-time Information

3 Approach and Implementation

In this section we will address, in the order given in the previous section, the LSP features we have implemented for the Truffle framework[2]. To support all the LSP features we describe, language implementations on Truffle need to implement the existing Truffle as well as a small number of new APImethods. Table 1 gives an overview of LSP support in different languages at the time of this writing.

Central to our approach is our code execution strategy for dynamic analysis. For each feature we detail both how information can be obtained statically and dynamically.

3.1 Obtaining Run-time Information

A detailed code completion for globals, locals, and propertiesof language elements is often implemented in IDEs using type information . For Code editor support for programming languages with typed static information about symbols, like Java, the required information can be easily obtained by parsing can be built by parsing and analyzing the source code. The situation for In dynamic languages with untyped symbols, like Python, is different, because the type depends on the associated run-time object only. There are however, there are only limited language-specific approaches to statically infer types of variables for such languages, but these approaches cannot be applied to information about the code. These approaches are necessarily limited by certain dynamic language features

[2] Changeset available at https://github.com/oracle/graal/pull/764

Table 1. An overview over selected Truffle languages and the current implementation status of GRAAL-LSP features.

LSP feature	SimpleLanguage (SL)	Python	FastR	GraalSqueak
Symbols	✓	✓	✓	
Goto-definition (local variables)	✓	✓		
Goto-definition (callables)	★	★	★	?
References and Highlights	✓	✓		?
Syntax Errors	✓	✓		✓
Signature Help		★		
Hover Information	★	★	★	★
Completion (globals)	✓	✓	✓	✓
Completion (locals)	✓	✓	★	✓
Completion (properties)	★	★		✓

✓ Available statically
★ Available after collecting run-time information
? Unclear how to implement

(e.g. *eval* or results of foreign function calls). Furthermore, we are not aware of a performance-focused language implementation framework ~~, which supports static type~~ that supports static inference in a language-agnostic way for their implemented dynamic languages. The RPython framework ~~uses~~ does use type inference internally~~to translate the source code written in the RPython language to a more efficient, statically typed representation. But this is only possible, because RPython is a subset of Python where some dynamic features are not supported. The static type inference for RPython can therefore not be easily~~ , but this cannot be applied to arbitrary dynamic languages developed with RPython.

~~Due to the described limitations of static type inference, we focus in our approach on how to enhance code completion for locals using run-time information. This approach will also build the basis for described in .~~

~~Approach and Challenges~~ ~~The basic idea,~~ Our basic approach to get run-time ~~values for locals~~ information at a specific position in the source code, is to execute that source code while observing the execution~~to collect run-time information at the desired position. This approach implies some preconditions. Language implementation frameworks need to provide a possibility to execute a program until a specific source code position is reached and then to inspect the current run-time state. This~~ . We cannot rely on any language specifics to analyze execution or attach to it, and we do not want to burden the developer with having to do significantly more work that is not related to simply running the language.

Observing the execution is already possible ~~, if~~ in Truffle, because the framework provides a debugger~~for the developed languages. If present, we~~ . We can reuse the same approach ~~as used by the debugger~~ to halt at ~~a breakpoint~~ an interesting location and obtain local symbols and their corresponding run-time values from the active frame. ~~To create the actual completion items, the symbols can be used as display names. Details , completion item kind, and documentation can be determined from the run-time values if the framework supports accessing this meta-information by querying the language implementations~~Details such as type, documentation, definition locations, and more can then be queried from Truffle objects.

Executing arbitrary source code requires to deal with unexpected and unwanted side-effects during execution, the possibility of a long running or non-terminating execution, or the fact that the desired position might not be reached by the execution flow.

Unreachable Source Sections ~~Depending on the source code, there might be~~ There are likely source sections which are unreachable by the normal execution flow of the current source code file. ~~In our opinion, this is the normal case for file-based programming languages. You typically split your program 's source code into multiple files, having~~ Typically a program has a limited set of ~~files, which define~~ entry points, ~~which then execute elements of the other files~~and only from those can other parts of the program be reached.

One ~~of these entry points are unit tests. Programmers typically write them to ensure the desired behavior of their programs and avoid regression of bugs. Therefore, unit testsshould cover most parts of the source code~~entry point to reach a (hopefully) large number of source sections are unit tests. Furthermore, most unit tests have a short execution time and are supposed to be free of unwanted side-effects – or at least have a clean-up routine included. The

systematic execution of unit tests is therefore a promising approach to get run-time information for code completion enhancement. This idea is inspired by *type harvesting* [7]~~, a practical approach to obtaining typing information in dynamic programming languages by exploiting unit tests. We only have to provide programmers a way~~. Programmers need to tell our language server how to execute the tests ~~. This could be done via a separate configuration file or~~ via a special comment in the first or last lines of a source code file, which encodes a path to a runnable script.

~~Unit tests might not be available, especially if you start writing a small script or do not follow Test-Driven Development (TDD) practices. Therefore, we suggest as fallback to try executing the current source code, in which the code completion was triggered, if no path to an entry point is defined.~~

~~Unsaved Changes~~ ~~Using the described approach~~ Our approach heavily favors test-driven development, because it will produce better results with it. It also favors writing unit tests that are small, finish quickly, and cover only a small part of the code. Especially when beginning to develop a system, ~~we have to care about unsaved changes. If we execute unit tests , we have to ensure that they do not import and execute files from the file system, if our server has newer, but yet unsaved versions. The LSP specifies no request sent from server to client to trigger saving files and we do not want to write the changes to the file systemfrom server-side, without asking the user. Therefore, we suggest to redirect file accesses of files with unsaved changes to (virtual) files which contain the changes. This could be realized by installing a custom file system, if the framework supports that. Otherwise, an enhanced code completion using run-time objects is only possible after saving all changes actively at client-side~~writing unit tests immediately while implementing small functions is the ideal scenario for our approach. However, unit tests might not be available, in which case we fall back to executing the current source code that is being edited.

Sandboxing the Execution Environment Users would not expect that the source code ~~,~~ they are currently working on ~~, will be~~ is executed when they trigger code completion, and they should not have to worry about side-effects in their code. Therefore, ~~we suggest to create a sandboxed execution environment~~any execution needs to happen in a sandboxed environment.

Our language server, because it can observe all execution, ~~when the executionof arbitrary source code is triggered implicitly. To create such a sandbox, the framework needs to provide an abstraction layer, which allows to observe and restrict all I/O of the implemented languages. This requirement is a superset of the one described to handle unsaved changes. If installing a custom file system is supported, the most radical approach would be to forbid all I/O in our sandboxed environment. Unfortunately, our approach of using run-time information for code completion would thereby be limited to theoretical examples only. Therefore, we suggest to allow at least some operations. For standard I/O, we suggest to allow any kind of input, but redirect or disallow all output. For file I/O, we suggest to simulate a read-only file system or, if supported, install a custom copy-on-write~~ is well positioned to provide such sandboxing. Furthermore, one of the goals of the Truffle framework is to be able to sandbox languages entirely, including all access to system resources. We use this to disallow all native access, as well as all access to sockets or the file system and redirect such requests as well as possible. Writing to the file system, ~~which copies each file before modification to a sandbox and which redirects further read and writes to that copies. For network I/O, we cannot make any assumptions concerning side-effects and suggest therefore to disallow all network~~ for example, instead writes to a in-memory filesystem that the server layers over the real filesystem on demand. Reading from files then first checks the in-memory file before going to the actual filesystem.

Network I/O~~. The same is required for calls~~ , signals, spawning new processes, or access to native libraries ~~.~~

~~The situation is different, if users explicitly allow side-effects. This might be the case if they have defined an entry point, like a unit testing script, which performs only known and uncritical side-effects. Therefore, we suggest to provide a configuration option, to tell our language server, if sandboxing shall be deactivated when collection~~ are not part of our approach and our server will just disallow such attempts. These limitations are difficult to lift in the general case, and we currently require well written unit tests with mock objects to replace these operations to gather run-time information~~for code completion~~.

~~Long Running or Non-terminating Executions~~Unsaved Changes ~~If the completion request takes too long to return a result, users might become impatient and distracted. Unfortunately, the~~ While editing, the source code is constantly out-of-sync with the actual file system. If we execute, we have to ensure that code imports do not hit the file system if our server has newer, but yet unsaved versions. The LSP specifies no request sent from server to client to trigger saving files.

As part of the sandboxing approach described above, we already provide an in-memory filesystem completely transparently to the languages and can serve the current state of unsaved files instead of the versions stored on the real filesystem.

Long Running or Non-terminating Executions The execution of arbitrary source code might take an unpredictable amount of time or might not finish at all~~, for example due to an endless loop or blocking operations. Furthermore,~~

the LSP does not support responding completion results incrementally. Therefore, the first thing we suggest is to explicitly stop execution when the execution flow our server immediately cancels execution when it has reached the desired position in the source code and we have collected the needed run-time objects to build our completion itemsinformation. Additionally, we suggest to add a general timeout to ensure an upper bound of execution executions are cancelled if they do not finish in a configurable amount of time. In case of a timeout, no run-time information can be provided to enhance our code completion, but an endless execution can be avoided.

Caching Run-time Information Executing a script or unit tests for every completion request might yield no results due to long running executions and timeouts. Therefore, we suggest to collect Truffle's instrumentation framework allows our language server to observer coverage of statements. If the user has defined multiple unit test entry points, we plan to use coverage information to determine which tests to (re-)run in order to gather run-time information for the whole source code once and cache this informationfor later completion requests. This collection run should be triggered manually by the users, e.g. by using custom commands, which the server can register during LSP initialization. These commands can then be made accessible at client-side via the *codeLens* request. This way, users can trigger the renewal of the cache manually, when they considered it reasonable. . This helps reduce the time required to produce results for the LSP client.

As a language-agnostic data structure for the cache, we suggest a mapping from lexical source sections of IR elements to collected

Caching Run-time Information Lag is in important factor in interactive code-editor. Our approach for gathering information relies heavily on the runtime required for executing the tests. Since the server already initializes the languages and the JIT compiler optimizes unit test executions to some extent, the execution times get better over time, but in our examples, we frequently had to wait 1-2 seconds to run the relevant test and get a result. We have yet to gather experience with larger test suites than a few hundred lines of code.

When run-time objects. To get a look-up key when a completion request arrives, we can again use the approach of finding an IR-element near the completion request's position, to get the IR-element's lexical source section. If the look-up succeedsinformation has been collected once e.g. for a function and the developer continues editing the function body, we can use the cached information to create the completion items.

The challenge of using such a lexical mapping is, that every time a *didChange* notification arrives, one or more sections of the source code will change. Depending on the change, the mappings in the cache might become invalid, but others might still be useful. We assume, that most edits will affect only mappings of the current local scope and in that scope only the edited line and following, so that all other mappings might be migrated. If the mapping keys reference source positions only via line number and character offset in that line, the migration could be done by simply adjusting the line number, if the changes add or remove one or more lines. Of course, we can only assume that the migrated mappings are still valid. The user has to trigger a new collection run manually when, for example, the execution flow is changed or variables are renamed.

3.2 Completion of Properties

The second kind of code completion we want to support is the .

shows a Python code snippet, containing a class with an instancevariable and a method . In the last line, we assume that the dot was the last character inserted and a code completion request has been triggered. We expect, that a completion result includes and as properties of the object referenced by the symbol . Other inherited properties, which all Python objects have, might also be included. still use information about parameters gathered in previous runs. For this we keep a mapping from cached run-time information to source locations on the path to the execution where we gather the information. We only invalidate if any of those locations change.

class Natural: n = 2

3.2 Implementation of LSP Features

def next(self): r = self.n self.n = self.n + 1 return r We will initially use SL[3] as a running example to show how our approach was implemented for that language, and then extend it to other Truffle languages. SL is Truffle's exemplary language implementation, it has first class functions, floating-point numbers and strings as primitive types and a single `object` type. Objects are implemented as simple key-value stores.

natural = Natural() natural. Like other Truffle tools, the LSP server is implemented as a `TruffleInstrument`. Figure 1 gives an overview of how this interacts with the Truffle framework. Instruments can ask for notifications in response to various events such as the creation of AST entry points (*call targets*) or the pending execution of an AST `Node` via one of its `execute*` methods. An instrument can also trigger parsing or execution via a `TruffleLanguage` instance, change the execution flow, and interact with any `TruffleObject` that flows through the AST. Finally, an instrument can influence the environment from which languages get access to OS level facilities including files and thus inject a custom `FileSystem` to serve requests for file

[3] https://github.com/graalvm/simplelanguage (accessed 2018-12-17)

access from from a sandboxed implementation rather than the real system.

To support this kind of code completion, we can build on the approach of obtaining and using run-time information described in the previous section. But there are some additional challenges which we have to solve. First of all, we have to detect that an object property completion shall be triggered instead of a completion of globals and locals. Second, the source code will typically contain a syntax error, when this kind of completion request is sent. Third, we have to identify the IR-element for which the completion of properties is desired. Fourth, we have to obtain the actual object associated with that IR-element, which might be available only at run-time. Finally, we need to get the object's properties to create the completion items.

Detecting an Object Property Completion As prerequisite for completion of properties, we need to be able to query a language implementation for its completion trigger characters. When a completion request arrives, we can then distinguish the two kinds of completion by comparing one or multiple characters directly in front of the completion request position with the trigger characters of the current language. If there is a match, we try a property completion. To support auto-activation of code completion, the trigger characters need to be sent to the client as part of the *initialize* responseSpecific to our instrument is that it launches an HTTP server on creation. This `LSPServer` handles the client's LSP requests. The handlers for the various requests are where the features we describe in this section are implemented. Whenever a file is touched, the `didOpen` and `didChange` callback uses the languages to `parse` the source. This immediately provides syntax errors and records any created AST nodes. These recorded nodes are then used in subsequent requests to map from editing locations to the AST, by filtering on the results of `getSourceSection`.

Finding the Desired Node ***Navigational Features*** To find an IR-element in front of the completion trigger character,

Symbols The *documentSymbol* and *symbol* requests of the LSP ask for known names and their locations on the source codeneeds to be parsable. We assume, that this is not always the case when a completion is triggered, because an inserted completion character might cause a syntax error. Therefore, we suggest to revert the last textual change which was sent from the client via the *didChange* notification and then try to parse the reverted source code again. Translated to our Python example in , the source code shown is no valid Python source code because of the last dot. Removing this dot makes the code parsable again. . For functions, this already works for all file-based Truffle languages. After parsing, our instrument was notified of any *root* node (e.g. of a function) and can use its `getSourceSection` method. The language developer has to implement this method, but this is already required for other Truffle tools and thus did not need additional support.

Now, we can search for a matching IR-element in front of the removed completion trigger character position, by checking its lexical source section. However, there might be multiple IR-elements whose lexical source sections end before the removed trigger character position. Furthermore, we are only interested in an IR-element which represents an expression, because we want to obtain its result. Hence, we assume again, that language implementations can expose meta-information of IR-elements through the framework, so that we can filter for example for expressions onlyWe have extended the list of tags that Truffle uses to identify the purpose of AST nodes with a *DeclarationTag*. For SL, we needed to extend the `hasTag` method in those nodes that introduce variables to respond true for that tag to have them show up in the list of symbols. For additional information, we also made use of the `getNodeObject` APIprovided by Truffle nodes. The convention we chose is that, if the language developer returns an object from this method for a node tagged as a declaration, that object can respond to the `kind` message with more detailed information about the declared variable. This allows languages to communicate statically known types if possible. For SL, we here return the primitive type or `object` if the RHS of the assignment is a constant.

Obtaining the Actual Properties To get the expression result, we need to execute the corresponding IR-element. In our Python example, we would evaluate an IR-element representing a variable read for the symbol . We can use the already described approach for obtaining run-time information for locals, to execute the source code until the desired IR-element is reached. Then, the framework needs to provide an API, so that we can intercept the result of the IR-element execution. Having the run-time object, we need the support of the framework to query the language implementation for properties of that object, together with further meta-information of these properties, to add details, kind, and documentation for each completion item.

We suggest some optimizations to avoid executing source code for every property completion request. First of all, we can optimize property completion for literals, if the framework supports identifying IR-elements as literal representations. The literal value is typically encoded in or referenced by the IR-element. If we can access this value

Go to definition When the user selects a function name and asks for the *definition* location, we can attempt to provide a static location or a dynamic one. In the static case, we only need to search the AST for a node representing the function call. These are already tagged in SL with the *CallTag* to support the Chrome debugger. From that node we match the

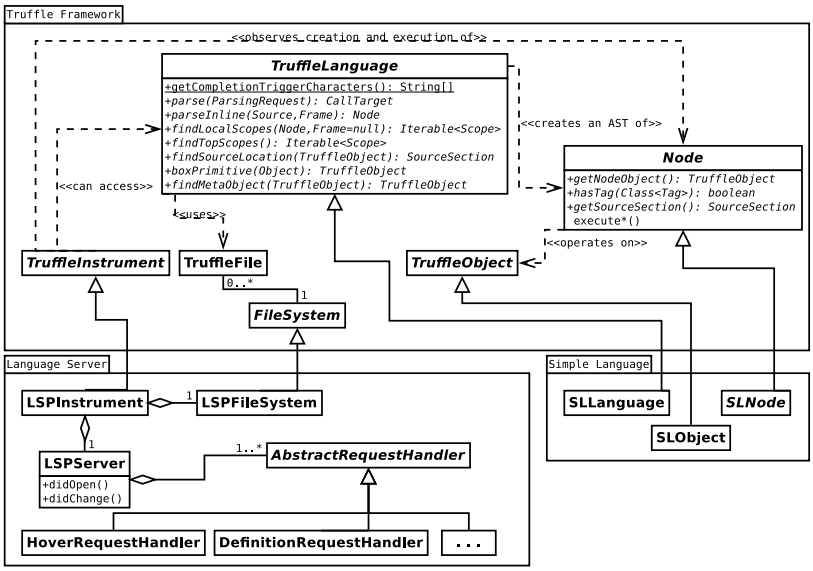


Figure 1. Simplified Class Diagram of LSP Components, Truffle, and SL

name of the function call to function definitions that have been parsed to show a list of candidates. Similarly, given our newly introduced *DeclarationTag*, we can ~~directly query for its properties~~also query the definition locations of variables with a certain name.

If ~~the IR-element is not representing a literal, we can check as next step if there are already cached~~ we are able to gather run-time information ~~available. If the framework and the language implementation support in-line evaluation, we can execute the IR-element in the context of the cached information.~~

~~If there are no cached run-time information available, we need to execute unit tests or~~ we can provide more accurate information. To gather runtime data, we send another `parse` request to the language and execute the resulting call target. This parse request either contains some unit test or other entry point declared by the user, or just the current source ~~code until the IR-element is reached, to get the required run-time object~~file as described in subsection 3.1. Without runtime data, we fall back to calling `findTopScopes` and looking for the name there.

~~If there is still no result, because the source position of the completion request might not be reachable or a timeout occurred, we can use a last fallback. We suggest to evaluate the IR-element in the global scope. This will at least provide a property completion for globals and context-independent expressions.~~

3.3 ~~Transferring the Approach to Other Requests~~

~~The approach of supporting code completion, described in the previous sections, includes parts that can be used to support further LSP requests. In our opinion, these additional requests we have chosen to highlight here are part of the essential feature set of an DE for dynamic languages.~~ We use the `TruffleLanguage#findSourceLocation` API, which language developers can implement to provide the definition location of objects where it is convenient to provide. For most languages including SL, function objects are supported by this APIand we can thus unambiguously determine the definition of a function call even when there are multiple functions with the same name.

References ~~In particular, we will highlight how to support~~ For the ~~*hover*~~*documentHighlight* ~~*, definition,*~~ and *references* ~~requests for callables and local variables, the *signatureHelp* request for callables, and the *documentHighlight* and *rename* requests for local variables only~~LSP messages, we re-use the CallTag and DeclarationTag annotations to find nodes with the names we are interested in. Furthermore, we ~~will highlight, how to support the~~ have added two more tags, ~~*symbol*~~*ReadVariableTag* and ~~*documentSymbolrequests for any kind of symbol declaration. As already described for code completion, we rely on meta-information exposed by the language implementation framework. This information allows us to search for certain kinds of IR-elements representing a call to a callable or an access to a variable at the requested position, or to retrieve all kind of symbol declarations in a source code file.~~ *WriteVariableTag*, which we have added to nodes representing local variable accesses in SL. Together, these allow us to return references for variables and functions.

To take scopes into account, we make use of another Truffle APIthat is already implemented in SL , `findLocalScopes`, which enumerates scopes from the inside out. If, for example, the client asks to highlight all



Figure 2. Diagnostics used to display a syntax error in SL.

references to a variable in a closure, the local scope APIwill enumerate surrounding scopes up to the top scope. Our server can then search for the same names in those scopes and find the correspondingly tagged notes. This APIcan be used statically or with a run-time `Frame` object, and returns lexical scope or dynamic scope nesting, respectively.

Informational Features

Document and Workspace Symbols The *documentSymbol* and *symbol* response items consist of a label, range, symbol kind, and container name. To provide this information, we assume that the language implementations can expose this information through the framework.

Detecting syntax errors First of all, we need to collect all IR-elements for a source code file, which define a symbol. Having them, label, range and symbol kind could be derived from the IR-element and its lexical source section. The container name could be derived from the names of the surrounding lexical scopes. To support the workspace-wide *symbol* request, we can build the union of *documentSymbol* requests for all files in the workspaceLanguage runtime parsers are not usually built to support incremental parsing or detection of more than one syntax error. SL, like other Truffle languages, fails parsing at the first syntax error, so that only one error per file can be sent to the client. By convention, a language syntax error is supposed to implement the `TruffleException` interface (not shown in the diagram), which provides a `getSourceLocation` method. Figure 2 displays how such a diagnostics notification is visualized in SL.

Signature Help and Goto-definition for Callables

Documentation and signature help The *signatureHelp* request is auto-activated at client-side, like code completion, if the server includes trigger characters for signature help in the *initialize* response. A *signatureHelp* response consists of a list of signatures, because there might be different variants, if the language implementation supports function overloading. Furthermore, the response includes two indices, telling which signature and which parameter should be shown in detail in the UI. Every signature in the list consists of a label, a list of parameters, and an optional documentation. A parameter also consists of label and documentation

If we can gather run-time information to access the object representing the function on the Truffle level, we have added two APImessages that the language developer needs to implement: `GetDocumentation` and `GetSignature`. These should return a documentation string or all parameter names and their documentation, respectively. The formatting of these is language specific and left to the language developer.

If we either cannot gather run-time information or the additional APIis not implemented by the language, we fall back to trying to find the definition of the function (as described above) and show the surrounding source code as a hint to the developer.

SL does not keep track of documentation strings in its parser, and neither does the parser keep parameter names separate from local variables. Implementing these two messages will require more work in the parser of SL.

To collect all the needed information, we suggest the following steps.

Hover information Find an IR-element representing a call to a We can support the *hover* request to show information for variables and callables with type information, provided there is run-time information available. If we have found an AST node representing a variable or callable at the requested positionExecute the source code until the IR-element is reached Intercept the callable run-time object represented by the IR-element Query the callable for label, parameters and documentation , we use `parseInline` and evaluate the resulting node. With the result, we build the hover response. Most Truffle languages including SL already implement the `findMetaObject` API. We use it on the result to show the type here in addition to the concrete value.

The first to third steps equal those for

Completion

We distinguish two kinds of code completions which need different levels of support by the language: completion of globals or locals and completion of properties.

Globals and locals We use the code snippet in Listing 1 to illustrate how to support code completion of properties. A challenge is, that the signature help should be displayed from the time, the trigger character is inserted, until the user has completed the signature. During that process, the source code might contain syntax errors, so that we have no IR-elements. Returning an empty response will end the displayed signature help. Therefore, this request might be supported only partial, depending on the particular language syntax.

Having obtained a callable object, the last step implies, that we can query the framework for information about that callable. Among all parameter names and their documentation, we need a formatted, language-specific representation of signature to create the label. If this is not supported by the framework, but we can get at least access to globals and locals. With the caret on line 3, a list of

parameter names, we suggest to use a Python-like syntax as signature label.

The approach to support the *definition* request for callables is quite similar. Only in the last step, we do not have to query the framework for signature information, but for the callable's definition location in the source code. Furthermore, we can apply a static search as fallback, if the dynamic approach does not yield a result. We can use the same approach as for the *symbol* request *completion* request is sent to the server for line 3 at character offset 4. The server searches the parsed nodes for the one with the nearest `SourceSection` and uses it to call `findTopScopes` and `findLocalScopes`. We would expect a completion result to include items for all SL globals (e.g. built-in functions) and the locals `param1`, to find all definitions of callables in the workspace and filter those matching the identifier of the callable at the requested position. For non-unique identifiers, this might yield many results, which are not related to the desired definition location. But in our opinion, it is better to provide the users a list of potential matches instead of providing nothing. `param2`, and `x`. The variable `y` should not be included, because no value is assigned to it until line 4.

Document Highlight, Rename and Find All References
The *documentHighlight*, *rename* and the *references* responses consist of a list of ranges in the source code. For the *documentHighlight* response, each range can additionally have an associated kind (Statically, we get the names and values of SL built-ins in the top scopes. The values are `SLObjects`, so we can use the normal `TruffleObject` messages to query if they are *callable*, , or), and for the *rename* response, each range needs an associated file Uniform Resource Identifier (URI) and the new text.

To support these requests for local variables, first of all a IR-element representing a read- or write-access of a variable at the requested positionneeds to be found. Having one, we suggest to search in the surrounding lexical scopes for all other IR-elements representing an access to variables with the same identifier. The required range can be obtained from the lexical source section of the IR-element. *instantiable*, and so on and return appropriate flags to the client. In languages that respond to documentation and signature messages for their objects, we show that here as well, but there is no such support for SL. For the *documentHighlight* response, the kind can be determined from the access kind of the variable. For the *rename* response, the URI is the one of the current file and the new text is already provided in the request.

Supporting *references* also for callables workspace-wide, we could use the same static fallback approach as described for the *definition* request. Due to potentially found false-positives, we do not suggest to also apply this static approach to the code-modifying *rename* request.

Listing 1. Completions in SL code

```
\DIFadd{function foo(param1, param2) }{
    \DIFadd{x = 42;
    _
    y = param1 + param2;
}}
\DIFadd{function main() }{ \DIFadd{foo("1", "2"); }}
```

Listing 2. An SL code snippet stopped at property completion

```
\DIFadd{function next(self) }{
    \DIFadd{r = self.n;
    self.n = self.n + 1;
    return r;
}}
\DIFadd{function Natural() }{
    \DIFadd{newobj = new();
    newobj.n = 2;
    newobj.next = next;
    return newobj;
}}
\DIFadd{function main() }{
    \DIFadd{natural = Natural();
    natural.
}}
```

`findLocalScopes`, SL statically returns the names of *all* variables defined.

Hover Information We can support the *hover* request for variables and callables, if there are cached run-time information available. If we have found an IR-element representing a variable or callable at the requested position, we can perform an in-line evaluation as described for code completion. Using the evaluation result, we can build the hover response, by querying the framework for information about In this case example we can just execute the entire file to gather run-time objects as described for code completion. The hover response consists of plaintext or markup content, so that we can put it together depending on the available information.

data. As soon as the execution reaches the node we are interested in, we stop and use the current live `Frame` in the `findLocalScopes` method. Now we get the local scope including live locals, we can show the types for `x` and the parameters and we reject `y`, because it is `null` at this point.

Property completion Listing 2 shows a code snippet where we define a `Natural` object with with a variable variable `n` and a method `next`. In the second-to-last line, we assume that the dot was the last character inserted and thus a code completion request was triggered. A completion result should `n` and `next` as properties of the object referenced by the symbol `natural`.

To support this kind of code completion, we need run-time information. We also have to detect that an object property completion is needed instead of a completion of

globals and locals. Second, the source code will typically contain a syntax error, when this kind of completion request is sent. Third, we have to identify the AST element for which the completion of properties is desired. Fourth, we have to obtain the actual object associated with that AST-element, which might be available only at run-time. Finally, we need to get the object's properties to create the completion items.

As prerequisite for completion of properties, we need to query the language implementation for its *completion trigger characters*. We have added the method `getCompletionTriggerCharacters` and language developers need to return a list of strings which should trigger completions. For SL, this is just the "." (dot).

When a completion request arrives and the location is preceded by a completion character, the source may not be parseable. Our current approach is to track the most recent modifications the client sent us since the last parseable state and use this state to execute the code. In many cases, this will simply be the state before the completion trigger character was inserted. In our example, just removing the last dot makes the code parseable again and we end up with the correct object to provide completion for. However, in some cases multiple edits have taken place in different parts of the source, and no parseable state can be reached this way that also leads to the desired code location. (We regard this problem as orthogonal to our approach, but it needs further investigation.)

4 ~~Implementation and Evaluation~~

Now, we search for a matching AST node in front of the removed completion trigger character position, by checking the source sections. We are only interested in the last node which represents an expression, because we want to obtain its result. Truffle's node tagging allows use to call `hasTag` with the `ExpressionTag` class to obtain only the expression `natural`.

For literals, the AST nodes can return the literal object directly for static completion via the `getNodeObject` method, where they should return a Truffle object with a member named `literal`. In our example, `natural` is not a literal, so we evaluate the expression node using `parseInline` in the current frame. If no run-time data is available, we attempt to evaluate in the global scope using `parse`. This way, at least completions on scope-independent objects can be satisfied.

~~We have implemented the language server using the existing Truffle instrumentation framework to observe and instrument execution and interaction with sources[4]. To support all the LSP features we described , language implementations on Truffle need to implement existing Truffle APIs.~~ Finally, if the returned object is not a `TruffleObject` object but a primitive type, we need to box it to read its members. We defined the API method `boxPrimitive` and implemented it in `SLLanguage`. Now we get a `SLObject` representing the primitive value and can query it for members.

~~Being implemented in an instrument, we can request parsing of sources (and report parsing errors), we can request static scope information at certain source code locations, and we can execute code at particular locations~~

Polyglot Code Editing Using SL code embedded in Python, we want to highlight how Truffle's support for polyglot programming works naturally with the features described above. Figure 3 shows Python code evaluating a SL code snippet which creates and returns a SL object. In the last ~~case, we can use any language objects observable during execution to query them about various properties as well as use the frameworks *Frame* concept to collect information about local scopes. To query the metadata we need, we propose to extend the default protocol for Truffle language objects to support querying signature and documentation, but our approach works even without these extensions. For some languages, these queries might be implemented with ease, but for others, some features might require fundamental changes in their implementations~~line, we get completions of properties for the foreign SL object. This demonstrates how the language agnostic approach of our server does not depend on the language an object originates from. After implementing our approach for SL , this example worked immediately after *only* adding the completion trigger character API to Python.

~~In this chapter~~

```
1  import polyglot
2
3  sl_code = """
4  function main() {
5      obj = new();
6      obj.nestedObj = new();
7      obj.nestedObj.func = main;
8      obj.nestedObj.value = 42;
9      return obj;
10 } """
11
12 sl_obj = polyglot.eval(string=sl_code, language="sl")
13 sl_obj.nestedObj.
```

```
func                <class 'int'>
value               of meta object: Object
```

Figure 3. Completion of properties for a foreign SL object in Python source code.

3.1 Application to Other Languages

In this section, we show by means of the Truffle language implementations Python, ~~SimpleLanguage,~~ FastR, and GraalSqueak that basic support of some LSP features can be achieved with minimal implementation effort in real languages. Furthermore, we explain ~~, why certain~~ which LSP

[4] ~~Changeset available at~~

features cannot be easily supported in some language implementations ~~. Finally, we describe~~ and other limitations of our approach~~and implementation of~~ .

3.2 ~~Evaluated Truffle Languages~~

Evaluated Truffle Languages ~~SimpleLanguage (SL)~~[4] ~~is Truffle's exemplary language implementation, to showcase Truffle features and help understanding its ideas and concepts. SL has first class functions. The type system only consists of primitive types and a single type. Objects are implemented as simple key-value stores. As showcase implementation , it implements most of the needed Truffle APIs so that, after implementing our proposed extensions with minimal effort, SL supports nearly all features.~~

The Python implementation for GraalVM[4] was modified to implement all APIwe proposed. For FastR[5]~~is a high-performance~~ , the implementation of the R[6] ~~programming languageused for statistical computing and graphics. Its goal is to become a drop-in replacement for the reference implementation GNU-R. Due to time constraints~~language, we have only implemented support for completion trigger characters~~yet, so that the FastR~~ , so the implementation is nearly unchanged. ~~Therefore, not many features are supported.~~

~~The Python implementation for GraalVM[7] is less complete than R and does not support many libraries yet, but is mature enough to demonstrate our approach.~~

This demonstrates that we can provide some support even with the least amount of language developer effort. GraalSqueak [12]~~is an early-stage experimental~~ , a Squeak/Smalltalk [9] implementation~~. In contrast to the other selected languages, it is an image-based, graphical live-programming environment. Furthermore, it uses the AST-based Truffle framework to implement a bytecode interpreter~~, is image- rather than file-based and executes bytecodes rather than a Smalltalk AST. Therefore, it is difficult to map concepts of GraalSqueak to ~~the file-based LSP protocol. Nonetheless, we~~ LSP. We have chosen it to show that at least some features can be supported with reasonable effort~~and to discuss limitations of our approach. We run GraalSqueak in headless mode to load an image and get access to its run-time objects. For now, we~~ . We treat every opened `st` file as an isolated Squeak/Smalltalk ~~workspace.~~

~~gives a broad overview over the features supported by these languages. In the following, we will highlight how well some of these features are already supported by showing and describing screenshots of the usage in a language client. We use Visual Studio Code to connect to an already running instance~~scope.

[4] ~~(accessed 2018-12-17)~~
[4] https://github.com/graalvm/graalpython (accessed 2018-12-17)
[5] https://github.com/oracle/fastr (accessed 2018-12-17)
[6] https://www.r-project.org (accessed 2018-12-17)
[7] ~~(accessed 2018-12-17)~~

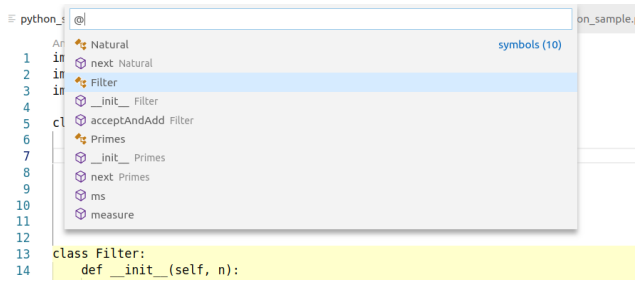


Figure 4. Symbol overview over the current Python document. The selected entry's source section is highlighted in the editor.

~~An overview over selected Truffle languages and their support of features. LSP feature SL GraalPython FastRGraalSqueakDiagnostics Completion (globals) Completion (locals) Completion (properties) Signature Help Hover Symbols Goto-definition (callables) Goto-definition (local variables) References (static) Highlighting variables *Requires run-time information ?Unclear~~

Navigational Features Both Python and FastR automatically support querying global symbols and navigating to them. For Smalltalk, this feature is not available, because the global symbols are not defined in any source file, but are part of the image. Thus, while we can list global symbols, we cannot offer a source location for them, which is required for the highlighting that LSP clients want to support, as seen in Figure 4.

To jump to definitions, we implemented the new DeclarationTag in Python to support jumping to the definition of local variables. For FastR, we did not change anything, but since the language supports returning the source location for functions, jumping to the definition of functions when runtime information is available worked immediately. In GraalSqueak, the DeclarationTag could enable jumping to the definition of local variables, but it's unclear how to implement jumping to function definitions at all. Futhermore, GraalSqueak works on bytecode, yet in Smalltalk source code, variables are declared explicitly at the beginning of blocks or methods. These declarations, however, are not available in the bytecode, where variables are only referenced by index. Thus, the Smalltalk implementation would have to do significant work to map back from AST nodes representing bytecode locations to source locations.

To fully support references, we had to add Write- and ReadVariableTags to Python. While FastR uses the CallTag, and thus we can provide references and highlights for functions, we cannot do the same for local variables. Additionally, scoping information in R relies on the dynamic execution extent, and thus we cannot properly support

~~Diagnostics used to display a syntax error in~~

```
import polyglot
                eval(path=, string=, language=, **kwargs)
polyglot.eval()
```

~~SL~~

Figure 5. Signature help for Python.

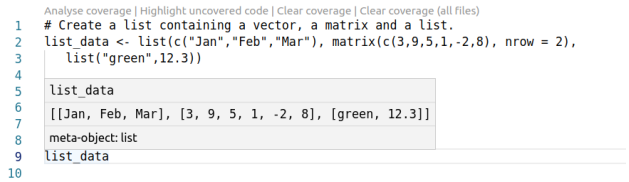


Figure 6. Displaying information on mouse hover for FastR source code using cached run-time information.

these features on FastR without both the tags and runtime information.

Informational Features

~~*Diagnostics*~~ All languages except FastR fail parsing at the first syntax error, so that only one error per file can be sent to the client. ~~displays how such a diagnostics notification is visualized in SL.~~ FastR does not throw a `TruffleException` in case of a syntax error, so that GRAAL-LSP cannot report any syntax errors for FastR. However, this is arguably a bug in the FastR implementation, as it also affects other tools such as the debugger.

Signature help and documentation requests are only fully implemented in Python. These use our newly introduced GetSignature and GetDocumentation messages. In Python, these were easy to implement since functions already store and expose their documentation to user code (Figure 5). For the implementation of these messages we thus just had to map them to the appropriate Python code. For Smalltalk classes and functions, we implemented the GetDocumentation message to show the last modified date, category, and author; these are stored as meta-data on these Smalltalk objects anyway.

~~*Code Completion*~~ Finally, hover information is available to the same extent as for SL in all three other languages, since they each support dynamic evaluation of source code in a given runtime frame. An example for FastR is shown in Figure 6.

Completion Figure 7 shows a code completion for globals and locals in ~~GraalPython~~Python. Completion items have different icons for classes, methods, and variables in the list of suggestions. For the selected item of a built-in class, a string representation of the corresponding meta-object is displayed, together with a documentation, if available. The same support is available ~~in SL (which can lexically provide local scopes), FastR (which needs run-time information for~~

~~Code completion~~

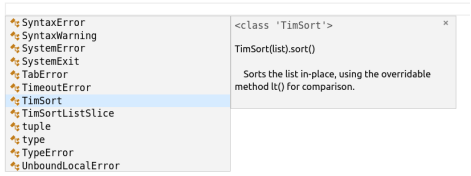


Figure 7. Completion of globals ~~and locals~~ with documentation in ~~GraalPython~~Python.

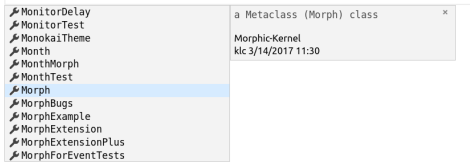


Figure 8. Code completion of globals in GraalSqueak.

~~locals),~~ for FastR and GraalSqueak (~~which only shows the globals available in the Smalltalk dictionary~~Figure 8).

The code completion of properties is supported to different extents in these languages. Run-time information are always required due to the dynamic nature of the languages. ~~Therefore, the source code needs to be parsable after removing the trigger character. For SL, supports the completion of properties, which are the keys of the underlying key-value store. For GraalPython, properties of objects can also be those inherited from their type hierarchy including fields and methods of their classes.~~ For FastR and GraalSqueak, we were not able to provide a completion of properties yet, although the Truffle ~~API~~ API allows it. These languages are missing an expression annotation on their AST nodes. ~~In GraalPython~~Implementing it will not only help the LSP support, but also add support for column breakpoints in the Chrome debugger, for example.

In Python, triggering a completion of properties for the code snippet `anObject.nestedObject`, by inserting a dot at the end, will yield completion items for all properties of the object referenced by `anObject.nestedObject`, because the nearest node tagged as expression is the one representing the whole source section. In SL, however, inserting a dot before the semicolon in the expressoin `anObject.nestedObject;` to trigger code completion, will yield no results, because the nearest node tagged as expression represents just the `nestedObject` string, and not `anObject.nestedObject`. This highlights that the developer has to take a bit of care designing their AST for optimal developer experience.

~~Using GraalPython and SL, we want to highlight, that Truffle supports polyglot programming and we can therefore provide completion of properties across language boundaries. shows Python code evaluating a SL code snippet which creates and returns a SL object. In the last line, we get completions of properties for the foreign SL~~

object, which would not be possible in SL code, as described in the previous paragraph.

The code completion request for properties is triggered fairly often in comparison to the other run-time information requiring requests. Unfortunately, the performance depends on the one hand on the execution time of the source code, which can be reduced by caching run-time information, but on the other hand it depends on the creation time of Truffle language contexts. We create a new one for each execution, which is quite cheap for SL, but for GraalPython, it takes a noticeable time. Reusing those contexts improves the performance, but introduces problems with cache invalidation and memory usage.

Completion of properties for a foreign SL object in GraalPython source code.

Signature Help Signature help is only supported with our proposed additional API, which we implemented for Python. can provide a formatted signature string as shown in . Implementing this message for example in SL might not be possible without changes to the language, because SL stores function arguments internally like any other local variable of the function, so that there is no knowledge left at run-time to obtain the signature of a function. This is an example where language developers need to store additional parse-time information to support more LSP features.

Signature help for GraalPython.

Hover Showing information on mouse hover is supported for variables and expressions of SL and GraalPython yet. For FastR, only variables can be accessed via their corresponding frame slot, to provide hover information. For all languages, cached run-time information needs to be available. A collection run can be triggered using the *codeLens* named *Analyse coverage*, as shown above the first line of code in . The displayed hover information includes a string representation of the variable's value and the meta-object.

Displaying information on mouse hover for FastR source code using cached run-time information.

Symbols As soon as a language eagerly creates AST nodes when the source code is parsed, can support the request for symbols by providing the names of all function root node instances known to the framework. Nodes can further be annotated as declarations, which we can use to choose symbol type and container name as shown for GraalPython in .

Symbol overview over the current GraalPython document. The selected entry's source section is highlighted in the editor.

Goto-definition and References Our goto-definition and find-all-references implementations depend on the supported tags of the Truffle language. The selected languages use the annotate nodes representing a call, so that goto-definition can be supported using run-time information. This works also across language boundaries, when using Truffle's polyglot API. Without run-time we can use a static symbol search. For GraalSqueak, which is not file-based, we do not know yet what a useful implementation of navigational requests might look like.

Variable Highlighting To support highlighting of variable accesses () we use nodes tagged as variable *reads* and *writes*. Here, the language developer needs to take care of assigning the correct source sections to their nodes for the highlighting to look correct. With correct assignments, we could also use these tags to support the *rename* refactoring for local variables.

Highlighting of variable accesses in GraalPython. The source sections of write accesses cover the whole assignment and are highlighted in a blueish shade.

4 Related Work

Our approach focuses on enabling performance-focused language implementation frameworks to provide their implemented dynamic languages with IDE-like tooling support with minimal effort. Another major part of our approach is based on collecting and using run-time information to enhance certain LSP features.

IDE-frameworks like Eclipse [4], Visual Studio[7], or IntelliJ[8] provide extension points to develop IDE-specific plugins for new languages or can be used as language clients if they implement the LSP. But as they do not support language developers in implementing programming languages in the first place and writing IDE-specific plugins or a language server still requires a lot of work, we do not further discuss them here. Furthermore, there are lots of language- and editor-specific tooling solutions for dynamic languages, which are also not our focus.

Language Workbenches such as like Spoofax [10], Xtext [6], or MPS [22] focus foremost on the efficient definition, reuse and composition of languages and their IDEs [5]. In contrast to the work presented here, performance and generality of the resulting languages is a secondary concern.

Visser et al. described an approach using the Spoofax language workbench [10] and the DynSem language [20] to systematically generate Truffle languages, i.e. AST interpreters, from DynSem specifications [21]. However, Spoofax provides plugins for the Eclipse IDE only. It would be interesting to see if LSP support can easily be enabled for Truffle ~~language~~ languages generated in this way.

~~Two approaches , which~~ Two approaches that focus on reusing existing tools of a live programming environment and the overall programming experience, are the Smalltalk-based IDEs Helvetia [14] and Squimera [13]. Helvetia enables embedding DSLs into Smalltalk, by adding hooks into the

[7] https://visualstudio.microsoft.com (accessed 2018-12-17)
[8] https://www.jetbrains.com/idea (accessed 2018-12-17)

Smalltalk compiler chain. This way, the DSLs can be translated on demand into Smalltalk code, so that existing tools of the environment still work, without the need to generate and start a new IDE. Squimera composes RPython interpreters into an RPython-based Smalltalk VM to re-use the Smalltalk tools for all composed languages. In contrast to this work, these approaches focus on Squeak/Smalltalk as the DE.

~~A major part~~ Part of our approach focuses on collecting and utilizing run-time information to answer LSP requests. There are a number of ~~interesting prior approaches to~~ prior approaches towards this idea.

The Hermion IDE [16] collects run-time information to enhance program comprehension by improving static source code navigation and browsing in Squeak/Smalltalk. Hermion builds upon partial behavioral reflection [17] to restrict the collection of run-time information to certain source code parts and detail levels. However, ~~due to performance issues, the users of the Hermion IDE have to make a trade-off between performance and the amount of collected information. The~~ users need to manually specify which classes shall be instrumented ~~,~~to keep the cost of instrumentation and reflective operations at an acceptable level. In Graal-lsp, the set of ~~AST nodes which shall be instrumented when collecting and caching run-time information,~~ instrumented AST nodes is implicitly defined by the LSP workspace, i.e. all parsed source code files. As ~~we collect snapshots of frames, we increase performance and reduce the amount of collected instances by taking only the first frame which is reported for every instrumented node. As~~ Hermion continuously observes the instrumented classes, it can collect different types for a variable, which the current Graal-lsp implementation ~~would not be able to detect~~does not do.

Holmes and Notkin [8] described an approach to improve the static find-references feature of the Eclipse IDE using collected run-time information. Only executed methods are included in the search results, so that users benefit from a limited result set related to their current application. To collect the required run-time information, they ~~use a custom AspectJ [11] aspect for~~ instrument the application's ~~test harness. Our prototype does not utilize~~ tests with AspectJ [11]. This is comparable to the case when run-time ~~information to answer the *references* request of the LSP yet. We perform a static symbol-based search in files of the current workspace, but it should be possible to enhance our implementation of the *references* request in a similar way~~data is available in our approach.

Type harvesting is another approach to collect type information for dynamic programming languages [7]. The solution is based on ~~a stepwise execution of bytecode while~~ observing the types of ~~relevant components~~variables during test execution. A similar feature exists in the PyCharm IDE which can collect ~~runtime~~ run-time information during a debug session. ~~Inspired by these approaches we also imagine unit tests as entry points for such type collection.~~ Graal-lsp has the advantage of running ~~the collection~~ with the JIT even when instrumented, yielding better performance than these approaches.

5 Conclusion and Future Work

~~In this work, we~~ We described the problem of language implementation frameworks lacking ~~IDE-like~~ tooling support for code editing, especially for implementations of dynamic programming languages. Language developers that use such frameworks to create and maintain programming languages with reasonable effort and good performance currently get at most some run-time tooling. We propose an approach to have IDE-like ~~tooling from the run-time information available in those frameworks~~ code assistance with minimal work required from the language developers. Our approach is based on a language-agnostic LSP server and collection and utilization of run-time information. As proof of concept, we have implemented Graal-lsp, a language server for the GraalVM.

~~Two~~ There are three limitations of our approach ~~are the use of language runtime parsers and the sandboxing while gathering run-time information. Since the runtime parsers typically stop parsing~~ to consider especially: First, since most runtime parsers stop on the first syntax error, we cannot report more than one syntax error per file to the client. ~~Furthermore~~Second, all LSP features rely on the availability ~~parsed IR to query, so the sourcecode always needs to be parsable for things like completion to work. Regarding the latter issue~~parseable source. Third, when our sandboxing prevents some kind of access or triggers a timeout, some source sections of interest may not be covered and thus no or only incomplete information is available.

Future work is two-fold: the main work will be to address the above limitations and find ways around them for realistic scenarios. This includes measuring how much time we can afford to spent executing code before the user gives up waiting for a result. The other is to cover more LSP features in our approach. Part of this means to investigate further how languages such as Squeak, which use bytecodes as part of the execution model and do not map to files, can be made to work with LSP. ~~We also think making our language server work better with polyglot programs must be part of this.~~

Despite these limitations, we already feel that our approach leads to a quickly usable ~~development environment from a language userpoint of view. On the other hand, it minimizes the burden on the side of language implementers as LSP now comes almost for free with Truffle. Furthermore,~~ code editor for the language user. Since we use the instrumentation APIof the Truffle framework to parse and execute source code and answer LSP requests, no separate parser or inference engine is required. Sandboxing avoids unwanted side-effects when executing arbitrary source code. As by-product of using the Truffle framework,

Graal-lsp supports polyglot editing assistance. With few APIextensions for Truffle languages to implement we demonstrated that a basic set of important LSP features can be supported with minimal implementation effort and we are confident that a similar approach ~~would~~ can work with reasonable effort in other, similar frameworks such as RPython.

References

[1] Davide Ancona, Massimo Ancona, Antonio Cuni, and Nicholas D Matsakis. 2007. RPython: a Step Towards Reconciling Dynamically and Statically Typed OO Languages. In *Proceedings of the 2007 Symposium on Dynamic Languages (DLS)*. ACM, 53–64. https://doi.org/10.1145/1297081.1297091

[2] Muhammad Asaduzzaman. 2018. http://hdl.handle.net/10388/8530. *Context-Sensitive Code Completion.* phdthesis. University of Saskatchewan. http://hdl.handle.net/10388/8530

[3] Carl Friedrich Bolz, Antonio Cuni, Maciej Fijalkowski, and Armin Rigo. 2009. Tracing the Meta-level: PyPy's Tracing JIT Compiler. In *Proceedings of the 4th workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems*. ACM, 18–25. https://doi.org/10.1145/1565824.1565827

[4] Jim des Rivières and John Wiegand. 2004. Eclipse: A platform for Integrating Development Tools. *IBM Systems Journal* 43, 2 (2004), 371–383. https://doi.org/10.1147/sj.432.0371

[5] Sebastian Erdweg, Tijs Van Der Storm, Markus Völter, Meinte Boersma, Remi Bosman, William R Cook, Albert Gerritsen, Angelo Hulshout, Steven Kelly, Alex Loh, et al. 2013. The State of the Art in Language Workbenches. In *International Conference on Software Language Engineering*. Springer, Springer International Publishing, 197–217. https://doi.org/10.1007/978-3-319-02654-1_11

[6] Moritz Eysholdt and Heiko Behrens. 2010. Xtext: Implement your Language Faster than the Quick and Dirty Way. In *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications (OOPSLA) 2010*. ACM, 307–309. https://doi.org/10.1145/1869542.1869625

[7] Michael Haupt, Michael Perscheid, and Robert Hirschfeld. 2011. Type Harvesting: a Practical Approach to Obtaining Typing Information in Dynamic Programming Languages. In *Proceedings of the 2011 ACM Symposium on Applied Computing*. ACM, 1282–1289. https://doi.org/10.1145/1982185.1982464

[8] Reid Holmes and David Notkin. 2010. Enhancing Static Source Code Search with Dynamic Data. In *Proceedings of 2010 ICSE Workshop on Search-driven Development: Users, Infrastructure, Tools and Evaluation*. ACM, 13–16. https://doi.org/10.1145/1809175.1809179

[9] Dan Ingalls, Ted Kaehler, John Maloney, Scott Wallace, and Alan Kay. 1997. Back to the Future: the Story of Squeak, a Practical Smalltalk Written in Itself. In *SIGPLAN Notices*, Vol. 32. ACM, 318–326. https://doi.org/10.1145/263700.263754

[10] Lennart CL Kats and Eelco Visser. 2010. The Spoofax Language Workbench: Rules for Declarative Specification of Languages and IDEs. In *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications (OOPSLA) 2010*, Vol. 45. ACM, 444–463. https://doi.org/10.1145/1869459.1869497

[11] Gregor Kiczales, Erik Hilsdale, Jim Hugunin, Mik Kersten, Jeffrey Palm, and William G Griswold. 2001. An Overview of AspectJ. In *European Conference on Object-Oriented Programming*. Springer, 327–354. https://doi.org/10.1007/3-540-45337-7_18

[12] Fabio Niephaus, Tim Felgentreff, and Robert Hirschfeld. 2018. GraalSqueak: A Fast Smalltalk Bytecode Interpreter Written in an AST Interpreter Framework. In *Proceedings of the 13th Workshop on Implementation, Compilation, Optimization of Object-Oriented Languages, Programs and Systems*. ACM, 30–35. https://doi.org/10.1145/3242947.3242948

[13] Fabio Niephaus, Tim Felgentreff, Tobias Pape, Robert Hirschfeld, and Marcel Taeumel. 2018. Live Multi-language Development and Runtime Environments. *The Art, Science, and Engineering of Programming* 2, 3 (mar 2018). https://doi.org/10.22152/programming-journal.org/2018/2/8

[14] Lukas Renggli, Tudor Gîrba, and Oscar Nierstrasz. 2010. Embedding Languages Without Breaking Tools. In *European Conference on Object-Oriented Programming*. Springer, 380–404. https://doi.org/10.1007/978-3-642-14107-2_19

[15] Armin Rigo and Samuele Pedroni. 2006. PyPy's Approach to Virtual Machine Construction. In *Companion to the 21st ACM SIGPLAN Symposium on Object-oriented Programming Systems, Languages, and Applications*. ACM, 944–953. https://doi.org/10.1145/1176617.1176753

[16] David Röthlisberger, Orla Greevy, and Oscar Nierstrasz. 2008. Exploiting Runtime Information in the IDE. In *Proceedings of the 16th IEEE International Conference on Program Comprehension, 2008. ICPC 2008.* IEEE, 63–72. https://doi.org/10.1109/ICPC.2008.32

[17] Éric Tanter, Jacques Noyé, Denis Caromel, and Pierre Cointe. 2003. Partial Behavioral Reflection: Spatial and Temporal Selection of Reification. *ACM SIGPLAN Notices* 38, 11 (2003), 27–46. https://doi.org/10.1145/949305.949309

[18] Michael L Van De Vanter. 2015. Building Debuggers and other Tools: We Can Have it All. In *Proceedings of the 10th Workshop on Implementation, Compilation, Optimization of Object-Oriented Languages, Programs and Systems*. ACM, 2. https://doi.org/10.1145/2843915.2843917

[19] Guido van Rossum. 1995. *Python Tutorial.* techreport CS-R9526. Centrum voor Wiskunde en Informatica (CWI).

[20] Vlad Vergu, Pierre Neron, and Eelco Visser. 2015. DynSem: A DSL for Dynamic Semantics Specification. In *26th International Conference on Rewriting Techniques and Applications (RTA) 2015 (Leibniz International Proceedings in Informatics (LIPIcs))*, Maribel Fernández (Ed.), Vol. 36. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 365–378. https://doi.org/10.4230/LIPIcs.RTA.2015.365

[21] Eelco Visser, Guido Wachsmuth, Andrew Tolmach, Pierre Neron, Vlad Vergu, Augusto Passalaqua, and Gabrieël Konat. 2014. A Language Designer's Workbench: A One-Stop-Shop for Implementation and Verification of Language Designs. In *Proceedings of the 2014 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software*. ACM, 95–111. https://doi.org/10.1145/2661136.2661149

[22] Markus Voelter and Vaclav Pech. 2012. Language Modularity with the MPS Language Workbench. In *Proceedings of the 34th International Conference on Software Engineering (ICSE), 2012*. IEEE, 1449–1450. https://doi.org/10.1109/ICSE.2012.6227070

[23] Christian Wimmer and Thomas Würthinger. 2012. Truffle: A Self-optimizing Runtime System. In *Proceedings of the 3rd Annual Conference on Systems, Programming, and Applications: Software for Humanity*. ACM, 13–14. https://doi.org/10.1145/2384716.2384723

[24] Thomas Würthinger, Christian Wimmer, Christian Humer, Andreas Wöß, Lukas Stadler, Chris Seaton, Gilles Duboscq, Doug Simon, and Matthias Grimmer. 2017. Practical Partial Evaluation for High-performance Dynamic Language Runtimes. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI) 2017*, Vol. 52. ACM, 662–676. https://doi.org/10.1145/3062341.3062381

[25] Thomas Würthinger, Christian Wimmer, Andreas Wöß, Lukas Stadler, Gilles Duboscq, Christian Humer, Gregor Richards, Doug Simon, and Mario Wolczko. 2013. One VM to Rule Them All. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software*. ACM, 187–204. https://doi.org/10.1145/2509578.2509581